DESENVOLVIMENTO MOBILE · MÓDULO 05 · TRILHA FLUTTER · NÍVEL INICIANTE

Widgets e Layouts

Árvore de widgets, componentes básicos, Row/Column/Stack, restrições de layout, listas, Material 3, temas e telas responsivas

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Explicar a interface declarativa, as árvores do Flutter e o caminho de um quadro.
- Usar textos, ícones, imagens, botões, cartões e campos de entrada.
- Montar telas com `Row`, `Column`, `Stack`, `Expanded` e espaçamentos.
- Entender a regra de restrições do layout e resolver os erros mais comuns.
- Exibir listas e grades eficientes com `ListView.builder` e `GridView`.
- Aplicar Material 3, temas claro e escuro, e adaptar a tela ao tamanho do aparelho.

Roteiro

Neste módulo

- Fundamentos: interfaces declarativas
- Tudo é widget
- Widgets básicos
- Layout
- Listas e grades
- Material 3, temas e responsividade
- Fechamento

Fundamentos: interfaces declarativas

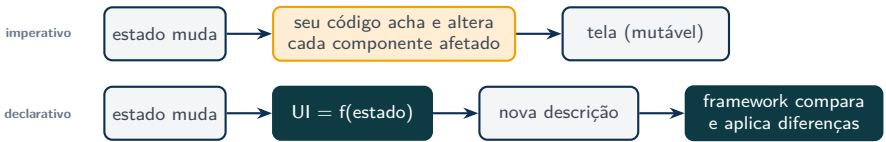
Interfaces imperativas e declarativas

Interface imperativa

O programa cria os componentes uma vez e depois os **altera** passo a passo (`setText`, `setVisibility`). Basta esquecer um deles para a tela mostrar dados errados.

Interface declarativa

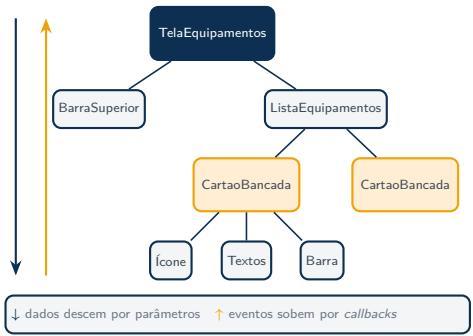
O programa **descreve** como a tela deve ser para cada estado. Quando o estado muda, o framework gera a nova descrição e aplica apenas as diferenças.



Um pouco de história

Views do Android e UIKit do iOS (2008) são imperativos. O React (2013) popularizou a UI declarativa, depois adotada pelo Flutter (2018), SwiftUI (2019) e Jetpack Compose (2021).

Composição: a tela como uma árvore



Composição

Construir interfaces complexas **combinando** componentes pequenos e independentes, em vez de herdar de componentes grandes. Cada componente recebe dados e devolve uma descrição de UI.

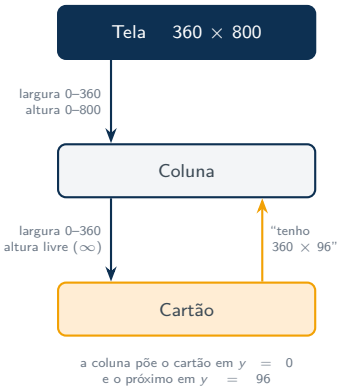
Analogia

Como peças de montar: o cartão é feito de ícone, textos e indicador; a lista, de vários cartões. Trocar uma peça não obriga a refazer as outras.

Um bom componente

Faz uma coisa só, recebe tudo por parâmetro, não sabe de onde vêm os dados e pode ser visto isoladamente em uma *preview*.

Layout por restrições



Restrições (*constraints*)

Intervalo de tamanhos que o pai aceita para o filho: largura e altura mínimas e máximas. **Justa** quando mínimo = máximo; **frouxa** quando o mínimo é zero.

O algoritmo em três frases

Restrições descem: o pai diz os limites. **Tamanhos sobem**: o filho escolhe um tamanho dentro deles. **O pai posiciona**: o filho não sabe onde está.

Uma passada só

Cada nó é medido uma única vez por quadro, o que mantém o layout rápido mesmo com milhares de nós. No Compose, cada `Modifier` ajusta as restrições antes de repassá-las; no Flutter, `Padding` e `SizedBox` fazem esse papel.

Do estado aos pixels: o caminho de um quadro



Quadro e orçamento de tempo

A tela é redesenhada no ritmo do sinal vertical do display (VSync): a 60 Hz cada quadro tem **16,7 ms**; a 120 Hz, só **8,3 ms**. Se o trabalho não couber, o quadro se perde e a animação engasga (*jank*).

Pular fases economiza tempo

Uma cor que anima só precisa de desenho; uma posição que muda, de layout e desenho. Ler o estado na fase mais tardia possível evita repetir as anteriores.

Dica

O *Performance* do Flutter DevTools e o *Layout Inspector* do Android Studio mostram onde o tempo do quadro foi gasto.

Por dentro: descrição, instância e renderização

Papel	Flutter	Compose
Descrição leve, recriada a cada mudança	`Widget`	chamada `@Composable`
Instância duradoura, guarda o estado	`Element` + `State`	composição (`remember`)
Mede, posiciona e desenha	`RenderObject`	`LayoutNode`
Informa leitores de tela	árvore de semântica	árvore de semântica

Analogia

O widget é a planta do arquiteto: barata de redesenhar. O elemento é o apartamento construído, onde mora o estado. O objeto de renderização é a estrutura medida e pintada. Mudar a planta não obriga a demolir o prédio.

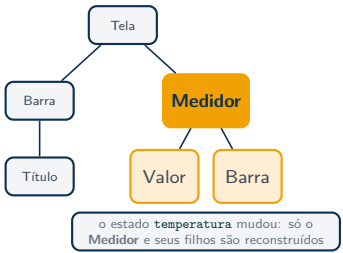
Por que separar

Descrições são criadas aos milhares por segundo, então precisam ser baratas. Medir e desenhar é caro, então os objetos de renderização são **reaproveitados** enquanto possível.

Semântica e acessibilidade

Em paralelo, o framework monta uma árvore de semântica (rótulos, papéis, ações) que o TalkBack e o VoiceOver leem. Ícone sem rótulo fica mudo para quem não enxerga (Módulo 04).

Atualizar sem refazer tudo



Flutter: *rebuild* e reconciliação

`setState` marca o elemento como sujo; no próximo quadro, só ele executa `build`. Filhos de mesmo tipo e chave são reaproveitados e só atualizados.

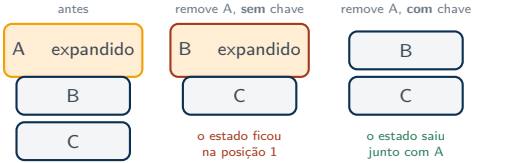
Compose: recomposição inteligente

O Compose sabe quais funções leram cada estado e só executa essas de novo; as que recebem parâmetros iguais aos anteriores são puladas (*skipping*).

Consequência importante

Funções de interface podem rodar muitas vezes, em qualquer ordem: devem ser **rápidas e sem efeitos colaterais**. Rede e banco ficam em APIs de efeito (Módulo 07).

Identidade e chaves



Compose

```
items(itens, key = { it.id }) { eq ->
  ItemEquipamento(eq)
}
```

Flutter

```
ItemEquipamento(
    key: ValueKey(eq.id), eq: eq, onTap: abrir)
```

Chave (*key*)

Identificador estável que diz ao framework qual item é qual entre duas versões da árvore. Sem chave, a correspondência é feita pela **posição**.

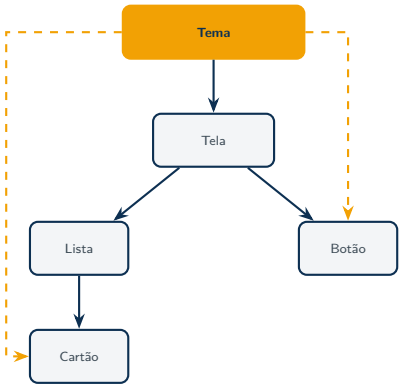
Quando importa

Em listas cujos itens têm estado próprio (expandido, campo digitado, animação) e podem ser inseridos, removidos ou reordenados.

Dica

Use um identificador do domínio, como o `id` do equipamento. Nunca o índice da lista nem um valor aleatório criado a cada reconstrução.

Temas, *tokens* e contexto implícito



tracejado: componentes leem o tema direto do contexto, sem receber parâmetros

Tokens de design

Decisões visuais nomeadas pelo **papel**, não pelo valor: `primary`, `surface`, `titleMedium`. O componente pede "a cor primária"; o tema decide qual é.

Contexto implícito

Tema, idioma e tamanho da tela ficam disponíveis para toda uma subárvore: `InheritedWidget` no Flutter (`Theme.of(context)`), `CompositionLocal` no Compose. Quem lê é atualizado quando o valor muda.

Dica

Com *tokens*, tema escuro e cor dinâmica (Android 12+) funcionam sem mexer nos componentes.

Pixels, densidade e telas de todos os tamanhos



160 dpi: 48 dp = 48 px



480 dpi: 48 dp = 144 px

Pixel independente de densidade

`px = dp × (dpi ÷ 160)`: um botão de 48 dp tem o mesmo tamanho físico em qualquer tela. No Flutter, é o pixel lógico (fator em `devicePixelRatio`).

Texto

Fontes também seguem a escala escolhida pelo usuário: `sp` no Android, `TextScaler` no Flutter.

Responsivo × adaptativo

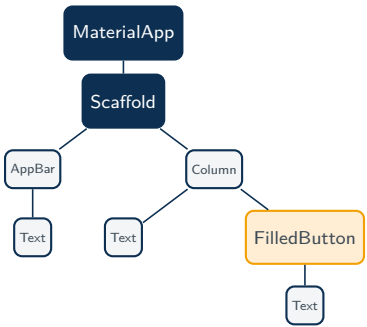
Responsivo: o mesmo layout se estica ou reorganiza. **Adaptativo**: um layout por classe de tamanho de janela (barra inferior no celular, trilho lateral no tablet). Pense em **espaço disponível**, não em modelo de aparelho.

Android 16 e telas grandes

Com `targetSdk` 36 (exigido pela Play desde agosto de 2026), telas com largura mínima $\geq$ 600 dp ignoram travas de orientação e de redimensionamento; no API 37 não haverá exceção.

Tudo é widget

A árvore de widgets



- Toda a interface é uma **árvore de widgets**: layout, estilo, gestos e até espaçamento são widgets.
- Um widget é uma **descrição imutável** de parte da tela.
- Quando o estado muda, o Flutter chama `build` de novo e compara a nova descrição com a anterior, redesenhando só o necessário.

UI = f(estado)

StatelessWidget e StatefulWidget

Sem estado

```
class CartaoBancada extends StatelessWidget {
  const CartaoBancada({
    super.key,
    required this.numero,
    required this.temperatura,
  });

  final int numero;
  final double temperatura;

  @override
  Widget build(BuildContext context) {
    return Card(
      child: ListTile(
        title: Text('Bancada $numero'),
        subtitle:
      Text('${temperatura.toStringAsFixed(1)} °C'),
      ),
    );
  }
}
```

Com estado (detalhes no Módulo 07)

```
class Contador extends StatefulWidget {
  const Contador({super.key});
  @override
  State<Contador> createState() => _ContadorState();
}

class _ContadorState extends State<Contador> {
  int cliques = 0;
  @override
  Widget build(BuildContext context) => FilledButton(
    onPressed: () => setState(() => cliques++),
    child: Text('Cliques: $cliques'),
  );
}
```

Dica

`BuildContext` é a posição do widget na árvore; serve para acessar tema, tamanho da tela e navegação.

Widgets básicos

Texto, ícones, imagens e botões

Componentes

```
Text(
  'Bancada 3 --- Osciloscópio',
  style: Theme.of(context).textTheme.titleMedium,
  maxLines: 1,
  overflow: TextOverflow.ellipsis,
),
const Icon(Icons.thermostat, color: Colors.orange),
Image.asset('assets/imagens/osciloscopio.png',
    width: 64, fit: BoxFit.cover),
Image.network(equipamento.fotoUrl, height: 120),

FilledButton(onPressed: reservar,
    child: const Text('Reservar')),
OutlinedButton(onPressed: () {},
    child: const Text('Cancelar')),
IconButton(onPressed: abrirMenu,
    icon: const Icon(Icons.more_vert),
    tooltip: 'Menu'),
```

Widget	Uso
`FilledButton`	ação principal
`OutlinedButton`	ação secundária
`TextButton`	ação discreta
`IconButton`	ícone clicável
`Card`, `ListTile`	itens de lista
`Chip`, `Badge`	etiquetas e contadores
`CircularProgressIndicator`	carregamento

Acessibilidade

Dê `tooltip` a botões de ícone e `semanticLabel` a imagens informativas.

Dica

`onPressed: null` desabilita o botão.

Campos de entrada e seleção

Formulário simples

```dart
final _form = GlobalKey<FormState>();
final _descricao = TextEditingController();

Form(
  key: _form,
  child: Column(children: [
    TextFormField(
      controller: _descricao,
      decoration: const InputDecoration(
        labelText: 'Descrição',
        border: OutlineInputBorder()),
      validator: (v) =>
          (v == null || v.isEmpty) ? 'Obrigatório' : null,
    ),
    SwitchListTile(
      title: const Text('Alimentação'),
      value: ligado,
      onChanged: (v) => setState(() => ligado = v)),
  ]),
);
// ao salvar: if (_form.currentState!.validate()) ...
```

TextEditingController

Lê e altera o texto do campo. Crie-o no `State` e libere em `dispose()`.

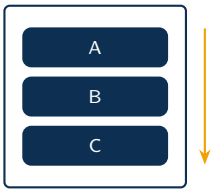
Widget	Valor
`Switch`, `Checkbox`	booleano
`Radio`, `SegmentedButton`	uma opção
`Slider`, `RangeSlider`	número, faixa
`DropdownMenu`	lista suspensa
`showDatePicker`	data

Dica

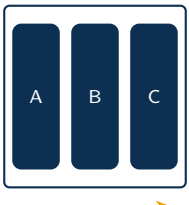
`keyboardType: TextInputType.number` abre o teclado numérico.

Layout

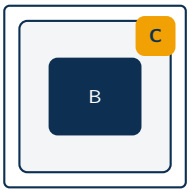
Row, Column e Stack



`Column` — vertical



`Row` — horizontal



`Stack` — sobreposição

Eixo principal (`mainAxisAlignment`)

`start`, `center`, `end`, `spaceBetween`, `spaceEvenly`; espaço fixo com `spacing: 8`.

Eixo cruzado (`crossAxisAlignment`)

`start`, `center`, `end`, `stretch`. Em `Stack`, posicione filhos com `Positioned` ou `Align`.

Compondo um item de lista

item_equipamento.dart

```dart
class ItemEquipamento extends StatelessWidget {
  const ItemEquipamento(
      {super.key, required this.eq, required this.onTap});
  final Equipamento eq;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) => Card(
    child: ListTile(
      onTap: onTap,
      leading: const Icon(Icons.memory),
      title: Text(eq.descricao,
          style: const TextStyle(fontWeight: FontWeight.bold)),
      subtitle: Text(eq.patrimonio),
      trailing: Icon(Icons.circle, size: 12,
          color: eq.disponivel ? Colors.green : Colors.orange),
    ),
  );
}
```

- `ListTile`: linha pronta com `leading`, `title`, `subtitle` e `trailing`.
- Para layouts próprios: `Row` + `Expanded` (ocupa o espaço restante) + `Padding`.
- `InkWell` adiciona toque com efeito de onda a qualquer widget.

Extraia widgets em classes

Prefira classes `StatelessWidget` a métodos que devolvem widgets: o Flutter consegue reaproveitar e evitar reconstruções.

Restrições na prática: erros comuns

Lista dentro de coluna

```
Column(
  children: [
    const CabecalhoResumo(),
    Expanded(            // ocupa o espaço que sobra
      child: ListView.builder(
        itemCount: itens.length,
        itemBuilder: (_, i) => ItemEquipamento(
            eq: itens[i], onTap: () => abrir(itens[i])),
      ),
    ),
  ],
)
```

Dica

O **Layout Explorer** do DevTools mostra as restrições de cada widget.

"Vertical viewport was given unbounded height"

Uma `ListView` dentro de uma `Column` recebe altura ilimitada e não sabe que altura ter. Envolva-a em `Expanded` ou dê uma altura fixa.

"A RenderFlex overflowed by 40 pixels"

O conteúdo de uma `Row` não coube (faixa amarela e preta). Use `Expanded`/`Flexible`, `Wrap` ou texto com `overflow`.

Listas e grades

ListView e GridView

lista_equipamentos.dart

```
ListView.separated(
  padding: const EdgeInsets.all(16),
  itemCount: itens.length,
  separatorBuilder: (_, __) => const SizedBox(height: 8),
  itemBuilder: (context, i) => ItemEquipamento(
    key: ValueKey(itens[i].patrimonio),
    eq: itens[i],
    onTap: () => abrir(itens[i]),
  ),
);

GridView.builder(
  gridDelegate:
      const SliverGridDelegateWithMaxCrossAxisExtent(
          maxCrossAxisExtent: 200, childAspectRatio: 1.2),
  itemCount: bancadas.length,
  itemBuilder: (context, i) =>
      CartaoBancada(numero: bancadas[i].id,
          temperatura: bancadas[i].temperatura),
);
```

`.builder` = sob demanda

Constrói só os itens visíveis. Uma `ListView(children: [...])` com mil itens cria todos de uma vez.

Keys

`ValueKey` com um identificador estável preserva o estado de cada item quando a lista é reordenada ou filtrada.

Dica

`RefreshIndicator`: "puxar para atualizar". `CustomScrollView` + *slivers*: cabeçalhos e listas combinados.

Material 3, temas e responsividade

Scaffold e tema

app.dart

```
MaterialApp(
  theme: ThemeData(
    colorScheme: ColorScheme.fromSeed(
        seedColor: const Color(0xFF0D3052)),
  ),
  darkTheme: ThemeData(
    colorScheme: ColorScheme.fromSeed(
        seedColor: const Color(0xFF0D3052),
        brightness: Brightness.dark),
  ),
  themeMode: ThemeMode.system,
  home: Scaffold(
    appBar: AppBar(title: const Text('LabMonitor')),
    floatingActionButton: FloatingActionButton(
        onPressed: novo, child: const Icon(Icons.add)),
    body: const ListaEquipamentos(),
  ),
);
```

Use o tema

`Theme.of(context).colorScheme.primary` e `textTheme.titleLarge` se adaptam ao tema escuro. Cores fixas como `Colors.red` não.

Cupertino

Para visual iOS, há os widgets `Cupertino*` e construtores `.adaptive` (ex.: `Switch.adaptive`), que mudam conforme a plataforma.

Dica

O pacote `google_fonts` aplica fontes do Google Fonts ao `textTheme`.

Telas responsivas

layout_adaptativo.dart

```dart
LayoutBuilder(builder: (context, c) {
  if (c.maxWidth >= 840) {          // expandido
    return Row(children: [
      const NavigationRail(...),
      const Expanded(child: ListaEDetalhe()),
    ]);
  }
  return const TelaCompacta();      // celular
});

final largura = MediaQuery.sizeOf(context).width;

SafeArea(child: conteudo);  // evita entalhe e barras
```

Dica

Unidades no Flutter são **pixels lógicos** (como o `dp`): o mesmo número tem o mesmo tamanho físico em telas de densidades diferentes.

Classe	Largura	Navegação
compacta	< 600	`NavigationBar`
média	600–839	`NavigationRail`
expandida	≥ 840	lista e detalhe lado a lado

Alvo de toque

Elementos clicáveis com pelo menos **48**×**48** pixels lógicos.

Fontes grandes

Teste com a fonte do sistema em 200 %: textos não podem ser cortados.

Fechamento

Resumo

Resumo

- A interface é uma árvore de widgets imutáveis; `build` descreve a tela para o estado atual.
- `StatelessWidget` só exibe; `StatefulWidget` guarda estado e chama `setState`.
- `Row`, `Column`, `Stack`, `Expanded` e `Padding` resolvem quase todo layout.
- Restrições descem, tamanhos sobem: listas em colunas precisam de `Expanded`.
- `ListView.builder` e `GridView.builder` para listas grandes, com `ValueKey`.
- `ColorScheme.fromSeed`, tema escuro e `LayoutBuilder` para telas adaptáveis.

Glossário (1/2)

Termo	Significado
UI declarativa	a interface é descrita como função do estado; o framework aplica as mudanças
Widget	descrição imutável e barata de uma parte da interface
`Element`	instância do widget na árvore; guarda o estado e o ciclo de vida
`RenderObject`	objeto que mede, posiciona e pinta
`BuildContext`	posição do widget na árvore; dá acesso a tema, tela e navegação
Rebuild	nova execução de `build` nos elementos marcados como sujos
Restrições	limites mínimos e máximos de largura e altura (`BoxConstraints`)
`Expanded`	filho de `Row`/`Column` que ocupa o espaço que sobra
Chave (`Key`)	identidade estável entre reconstruções (`ValueKey`, `GlobalKey`)

Glossário (2/2)

Termo	Significado
`InheritedWidget`	disponibiliza dados para toda uma subárvore (`Theme.of`)
***Token* de design**	decisão visual nomeada pelo papel (`primary`, `titleMedium`)
`ColorScheme.fromSeed`	gera a paleta Material 3 a partir de uma cor base
Pixel lógico	unidade independente da densidade (`devicePixelRatio`)
Classe de tamanho	classe de largura da janela: compacta, média ou expandida
Jank	quadro perdido por exceder o orçamento de tempo
Árvore de semântica	descrição da tela usada por leitores de tela como o TalkBack

Questões de revisão

1. Qual a diferença entre uma interface imperativa e uma declarativa? Por que a declarativa evita telas inconsistentes?
2. Explique a regra "restrições descem, tamanhos sobem, o pai posiciona" com um exemplo numérico.
3. Que papel têm as árvores de widgets, de elementos e de objetos de renderização?
4. O que acontece, passo a passo, depois de uma chamada a `setState`?
5. Por que o método `build` não deve iniciar uma requisição de rede?
6. Por que uma `ListView` dentro de uma `Column` gera o erro de altura ilimitada? Como resolver?
7. Quando é necessário passar uma `Key` aos itens de uma lista?
8. Por que usar `Theme.of(context).colorScheme.primary` em vez de uma cor fixa?

Exercícios

Exercício 1 — Layout

Reproduza um cartão de medidor: título, valor grande, unidade menor ao lado e uma `LinearProgressIndicator` colorida conforme a faixa de temperatura.

Exercício 2 — Erros de layout

Coloque uma `ListView` dentro de uma `Column` e uma `Row` com textos longos. Leia as mensagens de erro, use o Layout Explorer e corrija.

Exercício 3 — Responsivo

Faça a lista de equipamentos virar grade em telas largas e teste no navegador redimensionando a janela.

LabMonitor — Etapa 5

Implemente a tela **Equipamentos** (protótipo do Módulo 04): `Scaffold`, `AppBar`, FAB e `ListView.builder` de `ItemEquipamento`, com o tema azul-petróleo nos modos claro e escuro.

Para aprofundar

- Introdução aos widgets: `https://docs.flutter.dev/ui`
- Catálogo de widgets: `https://docs.flutter.dev/ui/widgets`
- Entendendo restrições: `https://docs.flutter.dev/ui/layout/constraints`
- Layouts: `https://docs.flutter.dev/ui/layout`
- Material 3 no Flutter: `https://docs.flutter.dev/ui/design/material`
- Design adaptativo: `https://docs.flutter.dev/ui/adaptive-responsive`

Respostas das questões de revisão

1. No imperativo, o código altera componentes existentes e precisa sincronizá-los a cada evento; no declarativo, descreve-se a tela para cada estado e o framework aplica as diferenças. Há um só caminho do estado à tela, então nada fica esquecido.
2. Na tela de 360 pixels lógicos, a `Column` passa ao cartão largura de 0 a 360 e altura livre; o cartão escolhe 360×96; a coluna o põe em $y = 0$ e o próximo em $y = 96$.
3. Widgets são descrições baratas, recriadas a cada mudança; elementos são as instâncias duradouras que guardam o `State`; objetos de renderização medem, posicionam e pintam, e são reaproveitados.
4. O elemento é marcado como sujo; no quadro seguinte seu `build` roda de novo; filhos de mesmo tipo e chave são reaproveitados e atualizados; depois vêm layout e pintura só do que mudou.
5. Porque `build` pode rodar muitas vezes por segundo: a requisição se repetiria a cada reconstrução. Ela pertence ao `initState`, ao gerenciador de estado ou ao repositório (Módulos 07 e 08).
6. A `Column` oferece altura ilimitada ao filho e a lista tenta ocupar toda a altura disponível. Envolva a lista em `Expanded` ou dê a ela uma altura definida.
7. Quando itens com estado próprio podem ser inseridos, removidos ou reordenados: a `ValueKey(id)` mantém o estado junto do item certo.
8. O *token* se adapta ao tema escuro, à cor dinâmica e a mudanças da identidade visual; a cor fixa, não.

Fim do Módulo 05

Próximo: Módulo 06 — Dart assíncrono: Future, async/await, Stream e Isolates.